E4 Educator v2.0

T16

I2S audio and MEMS microphone

Tier 2 · Tutorial 9 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How the I2S digital audio protocol works on the ESP32
- How the ICS43434 MEMS microphone on the E4 connects via I2S
- How to configure the ESP32 I2S peripheral to capture audio
- How to read audio samples and measure volume level
- What FFT is and how it transforms audio into frequency data
- How to compute a real-time FFT using the ESP32 DSP library
- How to display a live audio spectrum analyser on the TFT
- How to detect specific sound events — claps, tones, threshold crossings
- The E4 BANDSCOPE project — a complete real-time spectrum display

You will need:

- E4 Educator v2.0 board with TFT display and ESP32 fitted
- T08 completed — TFT sprite pipeline established
- T06 knowledge helpful — GPIO 32 shared with BLUE LED
- Button.h from previous tutorials

1 I2S and the ICS43434 MEMS microphone

I2S (Inter-IC Sound) is a serial audio protocol designed to carry digital audio between integrated circuits. Unlike analog microphones that output a voltage proportional to sound pressure, the ICS43434 MEMS microphone converts sound directly to a digital stream of 24-bit samples. This gives better noise immunity, more consistent results, and no ADC calibration concerns.

1.1 I2S signals on the E4

Signal	Define	GPIO	Purpose
BCLK	`MEMS_BCLK`	`GPIO 32`	Bit clock — clocks each audio bit. Shared with BLUE LED.
WS / LRCK	`MEMS_WS`	`GPIO 35`	Word select / left-right clock — switches between L and R channels each frame.
SD / DATA	`MEMS_SD`	`GPIO 34`	Serial data — the actual audio sample bits from the microphone.

GPIO 32 shared with BLUE LED

As noted in T06, GPIO 32 (MEMS_BCLK) is shared with the BLUE LED. When I2S is active, the LEDC output on GPIO 32 must be disabled. Do not use the BLUE LED in firmware that runs the MEMS microphone. Set LED_BLUE LOW and do not call ledcAttachPin(LED_BLUE, ...) if I2S is initialised on the same pin.

1.2 How the ICS43434 works

The ICS43434 is a bottom-ported MEMS microphone with a built-in 24-bit sigma-delta ADC. It outputs audio samples in I2S format, one 24-bit sample per WS cycle. The sample rate is determined by the BCLK frequency relative to the frame size.

The ICS43434 outputs its samples left-justified in a 32-bit I2S frame — the most significant 24 bits contain the sample, the bottom 8 bits are zero. The ESP32 I2S peripheral can be configured to right-shift these into a usable 24-bit value.

Parameter	ICS43434 detail
Sensitivity	−26 dBFS typical at 94 dB SPL
Frequency response	50 Hz – 20 kHz (±1 dB)
Sample depth	24-bit left-justified in 32-bit I2S frame
Supply voltage	1.65V – 3.6V. E4 runs at 3.3V.
Typical sample rate	8 kHz – 51.6 kHz. E4 course uses 16 kHz or 44.1 kHz.
L/R select	WS LOW = left channel. The ICS43434 on E4 is wired for left channel output.

2 Configuring the ESP32 I2S peripheral

The ESP32 has two I2S peripherals (I2S0 and I2S1). For the MEMS microphone in receive mode, we use I2S0 in standard Philips format with a 32-bit slot and 16-bit or 24-bit sample depth.

2.1 I2S library and configuration

```
#include <Arduino.h>
#include <driver/i2s.h>

#define I2S_PORT        I2S_NUM_0
#define I2S_SAMPLE_RATE 16000      // 16 kHz — speech quality
#define I2S_SAMPLE_BITS 32         // 32-bit slot (ICS43434 requirement)
#define I2S_DMA_BUF_COUNT 4        // Number of DMA buffers
#define I2S_DMA_BUF_LEN  256       // Samples per DMA buffer
#define SAMPLE_BUFFER_SIZE 512     // Samples to read per call

void setupI2S() {
  i2s_config_t cfg = {
    .mode             = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate      = I2S_SAMPLE_RATE,
    .bits_per_sample  = I2S_BITS_PER_SAMPLE_32BIT,
    .channel_format   = I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count    = I2S_DMA_BUF_COUNT,
    .dma_buf_len      = I2S_DMA_BUF_LEN,
    .use_apll         = false,
    .tx_desc_auto_clear = false,
    .fixed_mclk       = 0
  };

  i2s_pin_config_t pins = {
    .bck_io_num   = MEMS_BCLK,   // GPIO 32
    .ws_io_num    = MEMS_WS,     // GPIO 35
    .data_out_num = I2S_PIN_NO_CHANGE,
    .data_in_num  = MEMS_SD      // GPIO 34
  };

  i2s_driver_install(I2S_PORT, &cfg, 0, nullptr);
  i2s_set_pin(I2S_PORT, &pins);
  i2s_zero_dma_buffer(I2S_PORT);

  Serial.printf("I2S ready: %d Hz, 32-bit, left channel\n",
                I2S_SAMPLE_RATE);
}
```

Key point

The ICS43434 requires I2S_CHANNEL_FMT_ONLY_LEFT because it outputs only on the left channel (WS LOW). Using I2S_CHANNEL_FMT_RIGHT_LEFT would give you alternating valid and zero samples. Always use ONLY_LEFT for a single mono MEMS microphone on this configuration.

2.2 Reading audio samples

```
// Read a batch of 32-bit samples from I2S DMA
// The ICS43434 outputs 24-bit data left-justified in 32 bits
// Right-shift by 8 to get the useful 24-bit value

int32_t sampleBuffer[SAMPLE_BUFFER_SIZE];

size_t readI2SSamples() {
  size_t bytesRead = 0;
  i2s_read(I2S_PORT,
           sampleBuffer,
           sizeof(sampleBuffer),
           &bytesRead,
           portMAX_DELAY);

  size_t samplesRead = bytesRead / sizeof(int32_t);

  // Right-shift 8 bits to convert from left-justified 24-bit
  for (size_t i = 0; i < samplesRead; i++) {
    sampleBuffer[i] >>= 8;
  }
  return samplesRead;
}

// Measure RMS volume from sample buffer
float measureVolume(int32_t* samples, size_t count) {
  if (count == 0) return 0.0f;
  double sumSq = 0;
  for (size_t i = 0; i < count; i++) {
    double s = (double)samples[i];
    sumSq += s * s;
  }
  return (float)sqrt(sumSq / count);
}

// Convert RMS to approximate dB SPL
// Reference: ICS43434 full scale at 94 dB SPL = ~-26 dBFS
float volumeToDb(float rms) {
  if (rms < 1.0f) return 0.0f;
  // 24-bit full scale = 8388608
  float dbFS = 20.0f * log10f(rms / 8388608.0f);
  return dbFS + 94.0f + 26.0f;  // Approximate dB SPL
}
```

3 Real-time volume meter on TFT

Before tackling FFT, build a simple volume meter. This confirms the I2S configuration is correct and the microphone is capturing audio, and gives immediate visual feedback to encourage the learner.

```
#include <Arduino.h>
#include <driver/i2s.h>
#include <TFT_eSPI.h>
#include <math.h>
#include "Button.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);

#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_TEXT    0xFFFF
#define COL_LOW     0x07E0   // Green — quiet
#define COL_MID     0xFFE0   // Yellow — moderate
#define COL_HIGH    0xF800   // Red — loud
#define COL_LABEL   0xC618

#define I2S_PORT        I2S_NUM_0
#define I2S_SAMPLE_RATE 16000
#define SAMPLE_BUF_SIZE 512

int32_t samples[SAMPLE_BUF_SIZE];

// ... setupI2S() as in Section 2 ...

void drawVolumeMeter(float dbSPL) {
  spr.fillSprite(COL_BG);

  // Volume bar — full width of content zone
  int barMaxW = 220;
  float clampedDb = constrain(dbSPL, 30.0f, 90.0f);
  int barW = (int)map((long)clampedDb, 30, 90, 0, barMaxW);

  // Colour zones: green < 60dB, yellow 60-75dB, red > 75dB
  uint16_t barCol = (dbSPL > 75.0f) ? COL_HIGH :
                    (dbSPL > 60.0f) ? COL_MID  : COL_LOW;

  spr.drawRect(0, 20, barMaxW + 2, 40, COL_LABEL);
  if (barW > 0)
    spr.fillRect(1, 21, barW, 38, barCol);

  spr.setTextColor(COL_TEXT, COL_BG);
  spr.setTextSize(3);
  spr.setCursor(0, 80);
  spr.printf("%.1f dB", dbSPL);

  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setTextSize(1);
```

```
  spr.setCursor(0, 130);
  spr.print("Quiet           Loud");

  spr.pushSprite(10, 55);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("T16 Volume Meter  FW:%s\n", FW_VERSION);

  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init(); tft.setRotation(0); tft.fillScreen(COL_BG);  // Landscape 320x240
  ledcSetup(0, 5000, 8); ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  spr.setColorDepth(16);
  spr.createSprite(222, 160);

  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("MEMS Mic Volume");

  // Note: do not use LED_BLUE — GPIO 32 used by I2S BCLK
  setupI2S();
  Serial.println("Ready — make some noise!");
}

void loop() {
  size_t count = readI2SSamples();
  float rms   = measureVolume(samples, count);
  float dBSPL = volumeToDb(rms);
  drawVolumeMeter(dBSPL);
  Serial.printf("RMS: %.0f  dB: %.1f\n", rms, dBSPL);
}
```

Upload this and talk towards the MEMS microphone. The bar should respond immediately to your voice, rising into yellow and red zones when you speak loudly. Clap once close to the board — the bar should spike to red.

4 FFT — turning sound into frequency data

FFT stands for Fast Fourier Transform. It is a mathematical algorithm that converts a block of time-domain audio samples (a waveform) into a frequency-domain representation (a spectrum). The result tells you how much energy is present at each frequency — from bass to treble.

This is exactly what a spectrum analyser displays: the X axis shows frequency, the Y axis shows signal strength at that frequency. The FFT makes this possible in real time on the ESP32.

4.1 FFT fundamentals

Concept	Explanation
FFT size (N)	Number of samples used per FFT calculation. Must be a power of 2. Larger N = better frequency resolution but slower.
Frequency resolution	sample_rate / N. At 16kHz with N=512: 16000/512 = 31.25 Hz per bin. Each FFT output bin represents a 31.25Hz-wide frequency slice.
Frequency bins	FFT of N samples produces N/2 useful output bins (bins 0 to N/2-1). The upper half mirrors the lower.
Bin frequency	Bin k represents frequency k * (sample_rate / N). Bin 5 at 16kHz/512 = 5 * 31.25 = 156.25 Hz.
Nyquist limit	Maximum detectable frequency = sample_rate / 2. At 16kHz this is 8kHz. Human speech fits within this range.
Windowing	Multiplying samples by a window function (Hann, Hamming) before FFT reduces spectral leakage. Always apply a window.
Magnitude	Each FFT output bin has a real and imaginary component. Magnitude = sqrt(real² + imag²). This is what you display.

4.2 ESP32 DSP FFT library

The ESP32 Arduino core includes a hardware-accelerated DSP library (esp_dsp) with an optimised FFT implementation. Add it to platformio.ini:

```
lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  ; esp_dsp is part of the ESP-IDF — no separate install needed
  ; Include header directly:

; In main.cpp:
#include <esp_dsp.h>

; Alternatively use the arduinoFFT library for simpler API:
lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  kosme/arduinoFFT @ 1.6.2
```

For the E4 course, arduinoFFT is used for its simpler API. It is slightly slower than esp_dsp but produces identical results and is much easier to understand for a learner:

```
#include <arduinoFFT.h>
```

```
#define FFT_SIZE    512   // Must be power of 2
#define SAMPLE_RATE 16000

double vReal[FFT_SIZE];   // Real input samples
double vImag[FFT_SIZE];   // Imaginary (zero for real input)

arduinoFFT fft(vReal, vImag, FFT_SIZE, SAMPLE_RATE);

void computeFFT(int32_t* samples, size_t count) {
  // Copy samples into FFT input, normalise to -1.0 to +1.0
  size_t n = min(count, (size_t)FFT_SIZE);
  for (size_t i = 0; i < n; i++) {
    vReal[i] = (double)samples[i] / 8388608.0;  // 24-bit full scale
    vImag[i] = 0.0;
  }
  // Zero-pad if fewer samples than FFT_SIZE
  for (size_t i = n; i < FFT_SIZE; i++) {
    vReal[i] = 0.0; vImag[i] = 0.0;
  }

  // Apply Hann window to reduce spectral leakage
  fft.windowing(FFT_WIN_TYP_HANN, FFT_FORWARD);

  // Compute FFT
  fft.compute(FFT_FORWARD);

  // Calculate magnitude of each bin
  fft.complexToMagnitude();

  // vReal[0..FFT_SIZE/2-1] now contains magnitudes
  // vReal[0] = DC offset (ignore)
  // vReal[1] = 31.25 Hz bin
  // vReal[k] = k * (SAMPLE_RATE / FFT_SIZE) Hz
}

// Find the dominant frequency (peak bin)
float peakFrequency() {
  return fft.majorPeak();
}
```

5 Real-time spectrum analyser on TFT

This is the E4 BANDSCOPE — a real-time audio spectrum analyser that reads MEMS microphone audio, computes the FFT, and displays the frequency spectrum as a bar chart on the TFT. This is the most technically impressive project in the Tier 2 curriculum.

5.1 Spectrum display design

The spectrum display maps FFT bins to vertical bars across the TFT width. With 240 pixels wide and 512 FFT bins producing 256 useful bins, we group several bins per bar and display 60-80 bars total.

Parameter	Value for E4 BANDSCOPE
FFT size	512 samples
Sample rate	16,000 Hz
Frequency resolution	31.25 Hz per bin
Useful bins	256 (0 to 8,000 Hz)
Bars displayed	60 (bins 2-121, skipping DC)
Frequency range shown	62.5 Hz to 3,781 Hz (speech range)
Bar width	3 pixels each, 1 pixel gap = 4px per bar × 60 = 240px
Bar height	0 to 240px, log-scaled for visual balance

5.2 Complete BANDSCOPE firmware

```
#include <Arduino.h>
#include <driver/i2s.h>
#include <TFT_eSPI.h>
#include <arduinoFFT.h>
#include <math.h>
#include "Button.h"

// ── Display ──────────────────────────────────────────────
TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);

#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_GRID    0x1082
#define COL_BAR_LO  0x07E0   // Green — low frequencies
#define COL_BAR_MID 0xFD20   // Orange — mid frequencies
#define COL_BAR_HI  0xF800   // Red — high frequencies
#define COL_PEAK    0xFFFF   // White peak hold dots

// ── I2S / FFT ────────────────────────────────────────────
#define I2S_PORT        I2S_NUM_0
#define I2S_SAMPLE_RATE 16000
#define FFT_SIZE        512
#define USEFUL_BINS     (FFT_SIZE / 2)
```

```
#define NUM_BARS        60
#define FIRST_BIN        2   // Skip DC and sub-bass

double  vReal[FFT_SIZE];
double  vImag[FFT_SIZE];
int32_t samples[FFT_SIZE];

arduinoFFT fft(vReal, vImag, FFT_SIZE, I2S_SAMPLE_RATE);

// ── Display zones ──────────────────────────────────────────
#define SPEC_X    0
#define SPEC_Y   42
#define SPEC_W  240
#define SPEC_H  240
#define BAR_W     3
#define BAR_GAP   1

// ── Peak hold ──────────────────────────────────────────────
int peakHold[NUM_BARS] = {};
unsigned long peakTime[NUM_BARS] = {};
#define PEAK_HOLD_MS 1500

// ── Smoothing ──────────────────────────────────────────────
float prevBars[NUM_BARS] = {};
#define SMOOTH_FACTOR 0.3f   // 0=no smooth, 1=frozen

// ── I2S setup ──────────────────────────────────────────────
void setupI2S() {
  i2s_config_t cfg = {
    .mode             = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate      = I2S_SAMPLE_RATE,
    .bits_per_sample  = I2S_BITS_PER_SAMPLE_32BIT,
    .channel_format   = I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count    = 4,
    .dma_buf_len      = FFT_SIZE / 2,
    .use_apll         = false,
    .tx_desc_auto_clear = false,
    .fixed_mclk       = 0
  };
  i2s_pin_config_t pins = {
    .bck_io_num   = MEMS_BCLK,
    .ws_io_num    = MEMS_WS,
    .data_out_num = I2S_PIN_NO_CHANGE,
    .data_in_num  = MEMS_SD
  };
  i2s_driver_install(I2S_PORT, &cfg, 0, nullptr);
  i2s_set_pin(I2S_PORT, &pins);
  i2s_zero_dma_buffer(I2S_PORT);
}

// ── Chrome (draw once) ─────────────────────────────────────
void drawChrome() {
```

```
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("E4 BANDSCOPE");

  // Frequency axis labels
  tft.setTextColor(COL_LABEL, COL_BG);
  tft.setTextSize(1);
  tft.setCursor(0,  286); tft.print("63Hz");
  tft.setCursor(55, 286); tft.print("500Hz");
  tft.setCursor(110,286); tft.print("1kHz");
  tft.setCursor(168,286); tft.print("2kHz");
  tft.setCursor(210,286); tft.print("3.8k");
}

// ── Spectrum render ──────────────────────────────────────
void renderSpectrum(float* bars, int count) {
  unsigned long now = millis();

  spr.fillSprite(COL_BG);

  // Horizontal grid lines
  for (int g = 0; g < SPEC_H; g += 40) {
    spr.drawFastHLine(0, SPEC_H - g, SPEC_W, COL_GRID);
  }

  for (int i = 0; i < count; i++) {
    int barH = (int)bars[i];
    if (barH > SPEC_H) barH = SPEC_H;

    int x = i * (BAR_W + BAR_GAP);
    int y = SPEC_H - barH;

    // Colour by frequency position
    uint16_t col = (i < 15) ? COL_BAR_LO :
                   (i < 40) ? COL_BAR_MID : COL_BAR_HI;

    if (barH > 0)
      spr.fillRect(x, y, BAR_W, barH, col);

    // Peak hold
    if (barH >= peakHold[i]) {
      peakHold[i] = barH;
      peakTime[i] = now;
    } else if (now - peakTime[i] > PEAK_HOLD_MS) {
      if (peakHold[i] > 0) peakHold[i]--;
    }
    if (peakHold[i] > 0) {
      int py = SPEC_H - peakHold[i];
      spr.fillRect(x, py, BAR_W, 2, COL_PEAK);
    }
  }
```

```
  spr.pushSprite(SPEC_X, SPEC_Y);
}

// ── FFT and bar computation ──────────────────────────────
float barValues[NUM_BARS];

void computeAndDisplay() {
  // Read audio
  size_t bytesRead = 0;
  i2s_read(I2S_PORT, samples,
           FFT_SIZE * sizeof(int32_t), &bytesRead,
           portMAX_DELAY);

  size_t count = bytesRead / sizeof(int32_t);

  // Prepare FFT input
  for (size_t i = 0; i < FFT_SIZE; i++) {
    if (i < count)
      vReal[i] = (double)(samples[i] >> 8) / 8388608.0;
    else
      vReal[i] = 0.0;
    vImag[i] = 0.0;
  }

  // FFT
  fft.windowing(FFT_WIN_TYP_HANN, FFT_FORWARD);
  fft.compute(FFT_FORWARD);
  fft.complexToMagnitude();

  // Map FFT bins to display bars
  // Bins FIRST_BIN to FIRST_BIN+NUM_BARS*2
  // Using log scale for visual balance
  for (int b = 0; b < NUM_BARS; b++) {
    int bin = FIRST_BIN + b * 2;  // 2 bins per bar
    if (bin >= USEFUL_BINS) break;

    // Average two adjacent bins
    double mag = (vReal[bin] + vReal[bin+1]) * 0.5;

    // Log scale: convert to dB then map to bar height
    float dB = (mag > 0.0) ?
               20.0f * log10f((float)mag) : -80.0f;
    float barH = map((long)constrain((int)dB, -80, -10),
                     -80, -10, 0, SPEC_H);

    // Smooth bars to reduce flickering
    barValues[b] = prevBars[b] * SMOOTH_FACTOR +
                   barH * (1.0f - SMOOTH_FACTOR);
    prevBars[b] = barValues[b];
  }

  renderSpectrum(barValues, NUM_BARS);
```

```
  // Show dominant frequency in header
  float peak = fft.majorPeak();
  tft.fillRect(140, 12, 92, 16, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(1);
  tft.setCursor(142, 16);
  tft.printf("%.0f Hz", peak);
}

// ── Setup ──────────────────────────────────────────────────
void setup() {
  Serial.begin(115200);
  Serial.printf("E4 BANDSCOPE  FW:%s\n", FW_VERSION);

  // TFT init
  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init(); tft.setRotation(0); tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8); ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  // Sprite for spectrum zone
  spr.setColorDepth(16);
  spr.createSprite(SPEC_W, SPEC_H);

  drawChrome();
  setupI2S();
  Serial.println("BANDSCOPE running.");
}

// ── Loop ───────────────────────────────────────────────────
void loop() {
  computeAndDisplay();
  // No delay — I2S read blocks until buffer is full
  // This gives approximately 32ms per frame at 16kHz/512 samples
}
```

5.3 What to observe

- Silence: bars are at the noise floor, peak holds settle at low positions
- Speak: green bass bars (left) respond to low frequencies in voice, orange bars to mid-range consonants
- Whistle a note: a single bar spikes sharply showing the dominant frequency, with the frequency value updating in the header
- Clap: all bars spike simultaneously across the full spectrum
- Play music nearby: recognisable spectral patterns emerge — bass frequencies strong on the left, treble on the right
- Peak hold dots show the maximum recent level for each bar, falling slowly after 1.5 seconds

Try this

Whistle a steady note and slowly slide the pitch up. Watch the peak spike move from left to right across the spectrum display as your pitch rises. This is the FFT tracking the fundamental frequency of your whistle in real time — a genuinely satisfying demonstration of DSP in action.

6 Sound event detection

Beyond visualisation, the FFT output can be used to detect specific acoustic events. Clap detection, tone detection, and threshold-based alerting are common embedded audio applications.

6.1 Clap detection

A clap produces a broadband energy spike across all frequencies simultaneously. Detecting this is simple: compute the average magnitude across all bins and trigger when it exceeds a threshold:

```
float avgMagnitude(int firstBin, int lastBin) {
  double sum = 0;
  int count = lastBin - firstBin;
  for (int i = firstBin; i < lastBin; i++) {
    sum += vReal[i];
  }
  return (float)(sum / count);
}

#define CLAP_THRESHOLD  0.15f   // Adjust to taste
#define CLAP_COOLDOWN   500     // ms between detections

unsigned long lastClap = 0;

bool detectClap() {
  float broadband = avgMagnitude(2, 200);
  unsigned long now = millis();
  if (broadband > CLAP_THRESHOLD && now - lastClap > CLAP_COOLDOWN) {
    lastClap = now;
    return true;
  }
  return false;
}

// In loop() after computeAndDisplay():
  if (detectClap()) {
    Serial.println("CLAP detected!");
    digitalWrite(LED_GREEN, HIGH);
    delay(100);
    digitalWrite(LED_GREEN, LOW);
  }
```

6.2 Tone detection

A specific tone (like a 1kHz alert) produces energy concentrated in a narrow frequency band. Compare the energy in the target bin against adjacent bins to detect it:

```
// Detect a tone at approximately targetHz
bool detectTone(float targetHz, float threshold) {
  // Find the nearest FFT bin
  float binHz = (float)I2S_SAMPLE_RATE / FFT_SIZE;
  int targetBin = (int)(targetHz / binHz);
```

```
  targetBin = constrain(targetBin, 1, USEFUL_BINS - 2);

  // Energy at target vs neighbours
  double target  = vReal[targetBin];
  double below   = vReal[targetBin - 1];
  double above   = vReal[targetBin + 1];
  double avgNeighbour = (below + above) * 0.5;

  // Tone present if target is significantly above neighbours
  return (target > threshold && target > avgNeighbour * 3.0);
}

// Example: detect 1kHz tone
  if (detectTone(1000.0f, 0.05f)) {
    Serial.println("1kHz tone detected!");
  }
```

7 Performance notes and optimisation

Consideration	Guidance
FFT frame rate	At 16kHz, reading 512 samples takes 32ms. Maximum theoretical FFT frame rate is ~31fps. Including computation and display, expect 15–20fps.
Double precision	arduinoFFT uses double precision. Switching to float halves the computation time with negligible accuracy loss for audio display applications.
Sample rate vs range	Higher sample rate = higher frequency range but larger buffers and more computation. 16kHz covers speech well. 44.1kHz covers music to 22kHz.
FFT size vs resolution	Larger N = finer frequency resolution but more RAM and computation. N=512 at 16kHz gives 31.25Hz/bin — sufficient for speech analysis.
Smoothing	SMOOTH_FACTOR 0.3 gives a responsive but not jittery display. Increase towards 0.7 for a slower, smoother visual. Decrease towards 0.1 for maximum responsiveness.
WiFi + I2S	Running WiFi and I2S simultaneously is possible but WiFi interrupts can cause occasional I2S glitches. For best audio quality, disable WiFi or use a dedicated core with xTaskCreatePinnedToCore().

8 T16 summary and Tier 2 complete

Concept	Key takeaway
I2S protocol	Three wires: BCLK (bit clock), WS (word select), SD (data). Master generates BCLK and WS. ICS43434 outputs data on SD.
ICS43434	24-bit left-justified in 32-bit frame. Right-shift 8 bits after reading. ONLY_LEFT channel format. GPIO 32 shared with BLUE LED.
I2S config	I2S_MODE_MASTER | I2S_MODE_RX. I2S_BITS_PER_SAMPLE_32BIT. I2S_CHANNEL_FMT_ONLY_LEFT. I2S_COMM_FORMAT_STAND_I2S.

FFT	N samples → N/2 frequency bins. Frequency resolution = sample_rate/N. Apply Hann window before FFT to reduce leakage.
arduinoFFT	windowing() → compute(FFT_FORWARD) → complexToMagnitude(). vReal[k] = magnitude at bin k = k×(sample_rate/N) Hz.
Log scale display	Convert magnitude to dB: 20×log10(mag). Map dB range (-80 to -10) to bar height. Gives perceptually even display vs linear scale.
Peak hold	Track maximum bar height per bin. Hold for PEAK_HOLD_MS then decay. White dots show recent maximums.
Smoothing	Exponential moving average: new = prev×SMOOTH + current×(1-SMOOTH). Reduces bar flicker without losing responsiveness.
Clap detection	Broadband average magnitude exceeds threshold. Cooldown prevents multiple detections from one clap.
Tone detection	Target bin magnitude significantly exceeds adjacent bins. Peak must also exceed absolute threshold.

★ Tier 2 complete — all 9 tutorials done ★

You have covered TFT display, touch UI, SD logging, NVS settings, WiFi, MQTT, LittleFS, OTA updates, ESP-NOW sensor networks, and real-time audio spectrum analysis. Every E4 peripheral is now in your toolkit.

Tier 3 — five integrated projects — combines everything into complete real-world applications.

Tier 3 — Integrated projects preview

- P01 — Environmental monitor: AHT20 + DS18B20, TFT dashboard, MQTT to broker, OTA, NVS calibration — a complete deployable sensor station
- P02 — Data logger: multi-sensor, SD card CSV logging, RTC timestamp, LittleFS web dashboard — a professional data acquisition device
- P03 — Wireless sensor network: ESP-NOW star network, multiple E4 nodes, master TFT display, NVS provisioning — a complete IoT installation
- P04 — BANDSCOPE audio analyser: MEMS mic, I2S, FFT, real-time TFT display, MQTT peak data, clap/tone detection — a complete audio instrument
- P05 — Touch UI controller: multi-screen TFT app, touch navigation, NVS settings, OTA, MQTT control — a complete connected display device